

An Optimal Construction of Node-Disjoint Shortest Paths in Hypercubes

Cheng-Nan Lai

Department of Information Management,
National Kaohsiung Marine University, Kaohsiung, TAIWAN
cnlai@mail.nkmu.edu.tw

Abstract

Routing functions had been shown effective in constructing node-disjoint paths in hypercube-like networks. In this paper, by the aid of routing functions, m node-disjoint shortest paths from one source node to other m (not necessarily distinct) destination nodes are constructed in an n -dimensional hypercube, provided the existence of such node-disjoint shortest paths which can be verified in $O(mn^{1.5})$ time, where $m \leq n$. The worst-case time complexity and space complexity of the construction procedure are both $O(mn)$, which is optimal and hence improves previous results. In the situation that all of the source node and destination nodes are mutually distinct, experiment results show that the probability that there exist node-disjoint shortest paths (from the source node to the destination nodes) is greater than 87%, 89%, 91%, and 94% for $m=n=4, 5, 6$, and 7 , respectively.

1 Introduction

The hypercube network (hypercube for short) [11, 18, 19, 30, 31] is one of most popular interconnection networks (networks for short) because of its attractive properties, including regularity, node symmetric, link symmetric, small diameter, strong connectivity, recursive construction, partition capability, and small link complexity. An n -dimensional hypercube (abbreviated to an n -cube) [30] consists of 2^n nodes that are labeled with 2^n

binary numbers from 0 to 2^n-1 . Two nodes are connected by a link if and only if their labels differ by exactly one bit. Figure 1 shows the structure of a 4-cube where two disjoint 3-cube, i.e., a left 3-cube (consisting of eight nodes labeled with trailing 0) and a right 3-cube (consisting of eight nodes labeled with trailing 1), are connected by eight cross links. The diameter and connectivity of an n -cube are both n . The *connectivity* of a network is the minimum number of nodes whose removal can make the network disconnected or trivial [2].

Routing is a process of transmitting messages among nodes, and its efficiency is crucial to the performance of a network. Efficient and reliable routing can be achieved by using internally node-disjoint paths (disjoint paths for short), because they can be used to avoid congestion, accelerate transmission rate, and provide alternative transmission routes. Two paths are *internally node-disjoint* if they do not share any common node except their end nodes. The concept of disjoint paths arose naturally from the study of routing, reliability, and fault tolerance in parallel and distributed systems.

There are three categories of disjoint paths, i.e., one-to-one, one-to-many, and many-to-many [7]. Suppose that W is a network with connectivity n . According to Menger's theorem [2], there exist n disjoint paths between every two distinct nodes of W . They belong to the one-to-one category. Many one-to-one disjoint paths constructed for a variety of networks can be found in the literatures [6, 7, 9, 10, 20, 27, 32, 33].

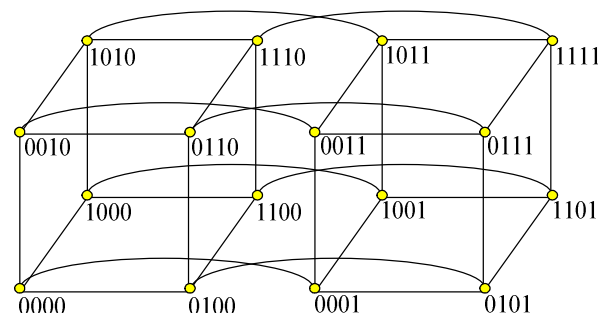


Figure 1. The structure of a 4-cube.

On the other hand, according to Theorem 2.6 in [1], there exist n disjoint paths from one node to other n distinct nodes in $\mathcal{W}$. They belong to the one-to-many category. One-to-many disjoint paths were first studied in [29] where the Information Dispersal Algorithm (IDA for short) was proposed on the hypercube. By taking advantages of disjoint paths, the IDA has numerous potential applications to secure and fault-tolerant storage and transmission of information. Some examples of one-to-many disjoint paths can be found in [3, 5, 8, 13, 17, 21-29]. Many-to-many disjoint paths (or named set-to-set disjoint paths), which connect two sets of nodes in $\mathcal{W}$, can be found in [4, 14-17, 19].

In order to reduce the transmission latency and cost, disjoint paths are desired to have their maximal length and total length minimized, respectively, where the *length* of a path is the number of links in the path. In this paper, we focus on a special problem of constructing m disjoint shortest paths from one source node to other m (not necessarily distinct) destination nodes in an n -cube, provided that such disjoint shortest paths exist. This problem was first studied in [13] where Gao et al. proposed an $O(mn^3)$ algorithm for solving it. Moreover, the algorithm was improved with respect to time complexity in [28] and [5] recently in case of $m=n$, and thus its worst-case time complexity had been reduced to $O(n^3 \log n)$ and $O(n^3)$, respectively.

Routing functions had been shown effective in constructing disjoint paths in hypercube-like networks [21-25]. In this paper, we show that by the aid of routing functions, m disjoint shortest paths from one source node to other m (not necessarily distinct) destination nodes can be constructed in an n -cube, provided the existence of such disjoint paths which can be verified in $O(mn^{1.5})$ time. The construction procedure has worst-case time complexity and space complexity both $O(mn)$, which improves the results of [5, 13, 28] and is optimal in the sense that any procedure must take at least $O(mn)$ time and space in the worst case to construct m disjoint paths in an n -cube. In order to figure out the probability that there exist disjoint shortest paths (from the source node to the destination nodes), an experiment was carried out for verifying each combination of mutually distinct source node and destination nodes (by executing a computer program). According to the experiment results, the probability is greater than 87%, 89%, 91%, and 94% for $m=n=4, 5, 6$, and 7 , respectively. Therefore, the construction of disjoint shortest paths is not only theoretically interesting but also useful in practical applications.

The rest of this paper is organized as follows: In the next section, routing functions are revisited and for a special routing function, a new construction procedure is proposed for constructing m disjoint shortest paths. The necessary and sufficient condition

for the existence of such disjoint shortest paths is shown in terms of routing functions, and can be verified in $O(mn^{1.5})$ time. Experiment results are described in Section 3. In Section 4, this paper concludes with some remarks on other applications of the construction procedure. For the brevity of this paper, whenever we discuss time complexity, we mean worst-case time complexity.

2 Construct all shortest disjoint paths with routing functions

Suppose that s is the source node and $d_1, d_2, \dots, d_m$ are m (not necessarily distinct) destination nodes in an n -cube, where $m \leq n$ and $s \notin \{d_1, d_2, \dots, d_m\}$. Since an n -cube is node symmetric, we assume $s =$

$\underbrace{00\dots 0}_n = 0^n$, i.e. the origin, without losing generality. Let $D = \{d_1, d_2, \dots, d_m\}$ be a multiset and $I = \{k_1, k_2, \dots, k_m\}$ be a set of m distinct integers ranging from 1 to n (actually, $k_1, k_2, \dots, k_m$ denote m dimensions of an n -cube). A *multiset* is a collection of elements in which multiple occurrences of the same element are allowed. In [21], a one-to-one correspondence $\Phi: D \rightarrow I$ was referred to as a *routing function*, and it was shown that routing functions can be effectively used to derive m disjoint paths from s to $d_1, d_2, \dots, d_m$, respectively, in an n -cube.

Throughout this paper, we let $e_{k_i} = 0^{k_i-1}10^{n-k_i}$ and use $d_{i,k}$ to denote the k 'th bit (from the left) of d_i , where $1 \leq i \leq m$, $1 \leq k \leq n$. That is, $d_i = d_{i,1}d_{i,2}\dots d_{i,n}$ is assumed. Intuitively, $\Phi(d_i) = k_i$ means that e_{k_i} is assigned as the immediate successor of s when we route from s to d_i . It is easy to see that e_{k_i} is contained in a shortest path from s to d_i if $d_{i,\Phi(d_i)} = d_{i,k_i} = 1$. Hence, a Φ satisfying $d_{i,\Phi(d_i)} = 1$ for all $1 \leq i \leq m$ is desired when we route from s to $d_1, d_2, \dots, d_m$. Unfortunately, such a Φ does not always exist for arbitrary D and I .

In the rest of this section, we first propose procedures for constructing m disjoint paths from s to $d_1, d_2, \dots, d_m$, respectively, in an n -cube. The procedures take routing function Φ as input, and it will be shown that if $d_{i,\Phi(d_i)} = 1$ for all $1 \leq i \leq m$, then the procedures can produce m disjoint shortest paths. Then, we show that the worst-case time complexity of the procedures is $O(mn)$. Finally, we give the necessary and sufficient condition, in terms of routing functions, for the existence of m disjoint shortest paths, which can be verified in $O(mn^{1.5})$ time (also shown in [5, 13, 28]).

2.1 The construction procedures

In order to take advantages of the partition

capability of an n -cube, a divide-and-conquer strategy is adopted by the construction procedures. According to the strategy, two sets of disjoint paths are first constructed in two disjoint $(n-1)$ -cubes, respectively, where the two $(n-1)$ -cubes are obtained by partitioning the n -cube according to a dimension chosen from routing functions. Then, by augmenting the two sets of disjoint paths with necessary cross links between the two $(n-1)$ -cubes, the resulting disjoint paths in the n -cube are obtained. Based on this construction method, a recursive procedure named *Paths0* is proposed in this section.

With inputs $\Phi, m, n, D=\{d_1, d_2, \dots, d_m\}$, and $I=\{k_1, k_2, \dots, k_m\}$, the procedure can produce m disjoint shortest paths, denoted by $P_1, P_2, \dots, P_m$, in an n -cube that connect $\{e_{k_1}, e_{k_2}, \dots, e_{k_m}\}$ and $\{d_1, d_2, \dots, d_m\}$, where P_i is the path to d_i for all $1 \leq i \leq m$. By augmenting $P_1, P_2, \dots, P_m$ with links $(s, e_{k_1}), (s, e_{k_2}), \dots, (s, e_{k_m})$, we have m disjoint paths from s to $d_1, d_2, \dots, d_m$, respectively. In the procedure, $*^{n-1}1$ and $*^{n-1}0$ represent two disjoint $(n-1)$ -cubes whose nodes have rightmost bits 1 and 0, respectively,

where $* \in \{0, 1\}$ and $*^{n-1} = \overbrace{** \dots *^{n-1}}$ (usually, an n -cube is represented with $*^n$). For each node $x=x_1x_2\dots x_n$ of an n -cube, define $x^{(n)}=x_1x_2\dots x_{n-1}(1-x_n)$, i.e., x differs from $x^{(n)}$ only in the bit x_n . We use $|d_i|$ to denote the number of bits 1 contained in d_i , i.e., the distance from s to d_i . The following is a formal description of the procedure.

Procedure *Paths0*(Φ, m, n, D, I).

- Step 1. If $n=2$, then {
 Construct $P_1, P_2, \dots, P_m$ as the m disjoint shortest paths in a 2-cube that connect $e_{\Phi(d_1)}, e_{\Phi(d_2)}, \dots, e_{\Phi(d_m)}$ and $d_1, d_2, \dots, d_m$, respectively.
 Return $P_1, P_2, \dots, P_m$. }
- Step 2. Determine d_c so that $|d_c| \geq |d_i|$ for all $1 \leq i \leq m$, where $1 \leq c \leq m$. If there are multiple candidates for d_c , then select arbitrary one. Without loss of generality, we assume $c=m$, $k_c=k_m=n$, and $\Phi(d_i)=k_i$ for all $1 \leq i \leq m$.
- Step 3. If $|d_m|=1$, then {
 Construct P_i as a shortest path from e_{k_i} to d_i for all $1 \leq i \leq m$.
 Return $P_1, P_2, \dots, P_m$. }
- Step 4. Partition D into D' and D'' , where $D'=\{d_i \mid d_{i,n}=0 \text{ and } 1 \leq i \leq m\}$ and $D''=\{d_i \mid d_{i,n}=1 \text{ and } 1 \leq i \leq m\}$. Without loss of generality, suppose $D'=\{d_1, d_2, \dots, d_r\}$ and $D''=\{d_{r+1},$

$d_{r+2}, \dots, d_m\}$, where $0 \leq r \leq m$.

- Step 5. Construct $P_1, P_2, \dots, P_m$ according to the following two cases.

Case 1. $d_{m,\alpha}=1$ for some $\alpha \in \{1, 2, \dots, n\} - \{k_{r+1}, k_{r+2}, \dots, k_m\}$.

/* Define $\Psi': \{d_1, d_2, \dots, d_r\} \rightarrow \{k_1, k_2, \dots, k_r\}$ as follows: $\Psi'(d_i)=\Phi(d_i)=k_i$ for all $1 \leq i \leq r$, and $\Psi'': \{d_{r+1}, d_{r+2}, \dots, d_m\} \rightarrow \{k_{r+1}, k_{r+2}, \dots, k_{m-1}, \alpha\}$ as follows: $\Psi''(d_m)=\alpha$ and $\Psi''(d_j)=\Phi(d_j)=k_j$ for all $r+1 \leq j \leq m-1$. */

Construct $P_1, P_2, \dots, P_r$ in $*^{n-1}0$ by executing *Paths0*($\Psi', r, n-1, \{d_1, d_2, \dots, d_r\}, \{k_1, k_2, \dots, k_r\}$). /* $P_1, P_2, \dots, P_r$ connect $\{e_{k_1}, e_{k_2}, \dots, e_{k_r}\}$ and $\{d_1, d_2, \dots, d_r\}$. */

Construct $P_{r+1}, P_{r+2}, \dots, P_m$ in $*^{n-1}1$ by executing *Paths0*($\Psi'', m-r, n-1, \{d_{r+1}, d_{r+2}, \dots, d_m\}, \{k_{r+1}, k_{r+2}, \dots, k_{m-1}, \alpha\}$). /* $P_{r+1}, P_{r+2}, \dots, P_m$ connect

$\{e_{k_{r+1}}^{(n)}, e_{k_{r+2}}^{(n)}, \dots, e_{k_{m-1}}^{(n)}, e_{\alpha}^{(n)}\}$ and

$\{d_{r+1}, d_{r+2}, \dots, d_m\}$. */

Augment $P_{r+1}, P_{r+2}, \dots, P_m$ with links $(e_{k_{r+1}}^{(n)}, e_{k_{r+1}}^{(n)}), (e_{k_{r+2}}^{(n)}, e_{k_{r+2}}^{(n)}), \dots, (e_{k_{m-1}}^{(n)}, e_{k_{m-1}}^{(n)}), (e_n, e_{\alpha}^{(n)})$.

Case 2. $d_{m,\alpha}=0$ for all $\alpha \in \{1, 2, \dots, n\} - \{k_{r+1}, k_{r+2}, \dots, k_m\}$.

/* Suppose that $\beta=k_r \neq n$ and $d_{m,\beta}=1$, where $r+1 \leq t \leq m-1$. Define $\Psi': \{d_1, d_2, \dots, d_r, d_m^{(n)}\} \rightarrow \{k_1, k_2, \dots, k_r, \beta\}$ as follows:

$\Psi'(d_m^{(n)})=\beta$ and $\Psi'(d_i)=\Phi(d_i)=k_i$ for all $1 \leq i \leq r$, and $\Psi'': \{d_{r+1}, d_{r+2}, \dots, d_{m-1}\} \rightarrow \{k_{r+1}, k_{r+2}, \dots, k_{m-1}\}$ as follows: $\Psi''(d_j)=\Phi(d_j)=k_j$ for all $r+1 \leq j \leq m-1$. */

Construct $P_1, P_2, \dots, P_r, P'_m$ in $*^{n-1}0$ by executing *Paths0*($\Psi', r+1, n-1, \{d_1, d_2, \dots, d_r, d_m^{(n)}\}, \{k_1, k_2, \dots, k_r, \beta\}$). /* $P_1, P_2, \dots, P_r, P'_m$ connect $\{e_{k_1}, e_{k_2}, \dots, e_{k_r}, e_{\beta}\}$ and $\{d_1, d_2, \dots, d_r, d_m^{(n)}\}$, where P'_m ends at $d_m^{(n)}$. */

Construct P_m as the combination of P'_m and the link $(d_m^{(n)}, d_m)$.

Construct $P_{r+1}, P_{r+2}, \dots, P_{m-1}$ in $*^{n-1}1$ by

executing $Paths0(\Psi'', m-r-1, n-1, \{d_{r+1}, d_{r+2}, \dots, d_{m-1}\}, \{k_{r+1}, k_{r+2}, \dots, k_{m-1}\})$. /*
 $P_{r+1}, P_{r+2}, \dots, P_{m-1}$ connect $\{e_{k_{r+1}}^{(n)}, e_{k_{r+2}}^{(n)}, \dots, e_{k_{m-1}}^{(n)}\}$ and $\{d_{r+1}, d_{r+2}, \dots, d_{m-1}\}$.
 */
 Augment $P_{r+1}, P_{r+2}, \dots, P_{m-1}$ with links
 $(e_{k_{r+1}}^{(n)}, e_{k_{r+1}}^{(n)}), (e_{k_{r+2}}^{(n)}, e_{k_{r+2}}^{(n)}), \dots, (e_{k_{t-1}}^{(n)}, e_{k_{t-1}}^{(n)}), (e_n, e_{k_t}^{(n)}), (e_{k_{t+1}}^{(n)}, e_{k_{t+1}}^{(n)}), \dots,$
 $(e_{k_{m-1}}^{(n)}, e_{k_{m-1}}^{(n)})$.

Step 6. Return $P_1, P_2, \dots, P_m$.

Intuitively, the m disjoint paths from s to $d_1, d_2, \dots, d_m$ can be obtained by first including m links $(s, e_{\Phi(d_1)}), (s, e_{\Phi(d_2)}), \dots, (s, e_{\Phi(d_m)})$ and then constructing m disjoint paths, i.e., $P_1, P_2, \dots, P_m$, that connect $\{e_{\Phi(d_1)}, e_{\Phi(d_2)}, \dots, e_{\Phi(d_m)}\}$ and $\{d_1, d_2, \dots, d_m\}$. The latter can be done by executing $Paths0(\Phi, m, n, D, I)$. In Step 1 of $Paths0(\Phi, m, n, D, I)$, we handle the trivial case when $n=2$. The n -cube will be divided into two $(n-1)$ -cubes $*^{n-1}0$ and $*^{n-1}1$ according to the dimension $\Phi(d_c)=n$, which is decided in Step 2. If $|d_m|=1$, i.e. $|d_i| \leq |d_m|=1$ for all $1 \leq i \leq m$, then the construction is simple in Step 3.

On the other hand ($|d_m| > 1$), D was partitioned into D' and D'' which contain nodes belonging to $*^{n-1}0$ and $*^{n-1}1$, respectively, as a consequence of Step 4. In Step 5, $P_1, P_2, \dots, P_m$ were constructed according to two cases in which paths in $*^{n-1}0$ and $*^{n-1}1$ need to be constructed in a recursive manner.

In Case 1, $|D'|$ disjoint paths in $*^{n-1}0$ that connect $\{e_{\Phi(d_1)}, e_{\Phi(d_2)}, \dots, e_{\Phi(d_r)}\}$ and D' , and $|D''|$ disjoint paths in $*^{n-1}1$ that connect $\{e_{\Phi(d_{r+1}}^{(n)}), e_{\Phi(d_{r+2}}^{(n)}), \dots, e_{\Phi(d_{m-1}}^{(n)}), e_{\alpha}^{(n)}\}$ and D'' were constructed. It should be noted that e_n in $*^{n-1}1$

corresponds to $s=0^n$ in $*^{n-1}0$ (refer to Figure 2). In Case 2, $|D'|+1$ disjoint paths in $*^{n-1}0$ that connect $\{e_{\Phi(d_1)}, e_{\Phi(d_2)}, \dots, e_{\Phi(d_r)}, e_{\beta}\}$ and $D' \cup \{d_m^{(n)}\}$, and $|D''|-1$ disjoint paths in $*^{n-1}1$ that connect $\{e_{\Phi(d_{r+1}}^{(n)}), e_{\Phi(d_{r+2}}^{(n)}), \dots, e_{\Phi(d_{m-1}}^{(n)})\}$ and $D'' - \{d_m\}$ were constructed. It will be shown later that $d_m^{(n)} \notin D'$. In both Case 1 and Case 2, the final paths were merged together in $*^n$ (n -cube) by augmenting some necessary cross links between $*^{n-1}0$ and $*^{n-1}1$.

For the integrity of our construction procedure, we give the procedure $paths1(\Phi, m, n, D, I)$, as formally described below, and the lemma followed.

Procedure $Paths1(\Phi, m, n, D, I)$.

- Step 1. Construct $P_1, P_2, \dots, P_m$ by executing $Paths0(\Phi, m, n, D, I)$. /* $P_1, P_2, \dots, P_m$ connect $\{e_{k_1}, e_{k_2}, \dots, e_{k_m}\}$ and $\{d_1, d_2, \dots, d_m\}$, where P_i ends at d_i for all $1 \leq i \leq m$. */
- Step 2. Augment $P_1, P_2, \dots, P_m$ with links $(s, e_{k_1}), (s, e_{k_2}), \dots, (s, e_{k_m})$.
 Construct Q_i as P_i for all $1 \leq i \leq m$.
- Step 3. Return $Q_1, Q_2, \dots, Q_m$.

Lemma 1. Given a routing function $\Phi: D \rightarrow I$ such that $d_{i, \Phi(d_i)} = 1$ for all $1 \leq i \leq m$, then $Paths1(\Phi, m, n, D, I)$ can produce m disjoint paths $Q_1, Q_2, \dots, Q_m$ from s to $d_1, d_2, \dots, d_m$, respectively, so that Q_i is shortest with length $|d_i|$ for all $1 \leq i \leq m$.

For the proof of Lemma 1, the interested reader may refer to [22].

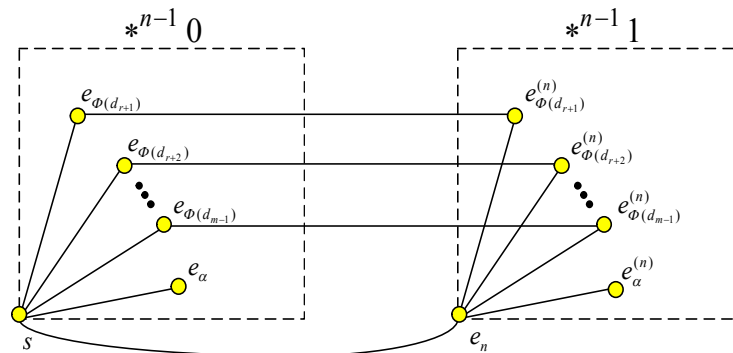


Figure 2. e_n in $*^{n-1}1$ corresponds to $s=0^n$ in $*^{n-1}0$.

2.2 The time complexity of $Paths1(\Phi, m, n, D, I)$

In this section, we first show that the time complexity of $Paths1(\Phi, m, n, D, I)$ is $O(mn)$ in the worst case, and then give the necessary and sufficient condition, in terms of routing functions, for the existence of m disjoint shortest paths. Since $Paths0(\Phi, m, n, D, I)$ dominates the execution of $Paths1(\Phi, m, n, D, I)$, it suffices to show that the time complexity of $Paths0(\Phi, m, n, D, I)$ is $O(mn)$ in the worst case. In $Paths0(\Phi, m, n, D, I)$, Step 1 takes constant time and all of Step 2, Step 3, and Step 4 take $O(m)$ time. The execution of Step 5 (excluding the recursive calls) takes $O(m)$ time because the condition " $d_{m,\alpha}=1$ for some $\alpha \in \{1, 2, \dots, n\} - \{k_{r+1}, k_{r+2}, \dots, k_m\}$ " in Case 1 of Step 5 can be determined in $O(m)$ time, as explained below. Initially, for node d_i , we create an array $DL(d_i)$ which consists of n elements each corresponding to a dimension, where $1 \leq i \leq m$. Each element has one data and two references which may refer to other elements. The data of $DL(d_i)[k]$ is k if $d_{i,k}=1$, and 0 otherwise ($d_{i,k}=0$), where $1 \leq k \leq n$ and $DL(d_i)[k]$ denotes the k 'th element of $DL(d_i)$. The references of elements of $DL(d_i)$ form a doubly-linked list, denoted by $DL(d_i).list$, where the data of each element is not zero. That is, each dimension k such that $d_{i,k}=1$ is stored in $DL(d_i).list$.

For an illustrative example, refer to Figure 3(a), the initial structure of $DL(d_i)$ for $d_i=10101$ was shown. The creation of $DL(d_i)$ for all $1 \leq i \leq m$ takes $O(mn)$ time. There are two operations, i.e., "delete" and "insert", for deleting dimension from and inserting dimension into $DL(d_i).list$, respectively. For deleting dimension k , the element $DL(d_i)[k]$ is removed from $DL(d_i).list$ and the length of $DL(d_i).list$, denoted by $DL(d_i).length$, decreases by 1 if $DL(d_i)[k] \in DL(d_i).list$, where $1 \leq k \leq n$. For an illustrative example, refer to Figure 3(b), the structure of $DL(d_i)$ for $d_i=10101$ was changed after deleting dimension 3. On the other hand, for inserting dimension k , the element $DL(d_i)[k]$ is inserted into $DL(d_i).list$ as the first element and $DL(d_i).length$ increases by 1 if $DL(d_i)[k] \notin DL(d_i).list$ and the data of $DL(d_i)[k]$ is not zero. For an illustrative example, refer to Figure 3(c), after inserting back dimension 3, the resulting $DL(d_i).list$ for $d_i=10101$ has $DL(d_i)[3]$ as the first element. It is not difficult to check that the "delete" and "insert" operations both take constant time.

By the aid of $DL(d_m)$, the condition " $d_{m,\alpha}=1$ for some $\alpha \in \{1, 2, \dots, n\} - \{k_{r+1}, k_{r+2}, \dots, k_m\}$ " can be determined by first deleting dimensions $k_{r+1}, k_{r+2}, \dots, k_m$ from $DL(d_m).list$, and then checking the value of $DL(d_m).length$.

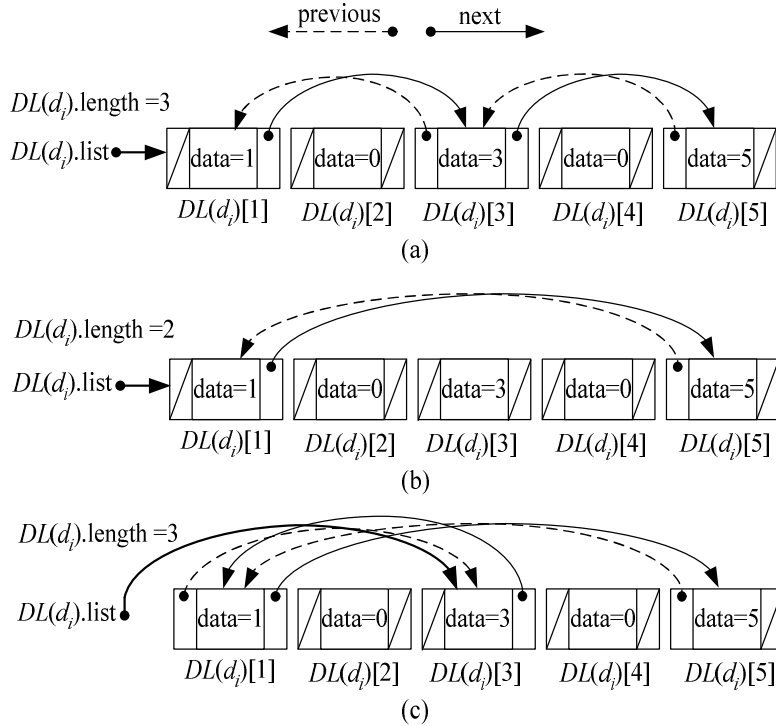


Figure 3. (a) The initial structure of $DL(d_i)$ for $d_i=10101$.
 (b) The structure of $DL(d_i)$ after deleting dimension 3.
 (c) The structure of $DL(d_i)$ after inserting back dimension 3.

If $DL(d_m).length > 0$, then the data of the first element of $DL(d_m).list$ is the desired α . Finally, insert dimensions $k_m, k_{m-1}, \dots, k_{r+1}$ back into $DL(d_m).list$ to recover $DL(d_m).list$. Hence, the determination takes $O(m-r) \leq O(m)$ time. We note that in order to maintain correct $DL(d_i)$ for all $1 \leq i \leq m$ during execution, dimension $k_m = n$ must be deleted from $DL(d_i).list$ (and $DL(d_m)$ is assigned to $DL(d_m^{(n)})$) before recursive calls, and then inserted into $DL(d_i).list$ (and $DL(d_m^{(n)})$ is assigned back to $DL(d_m)$) after recursive calls (in $O(m)$ time).

An n -cube can be partitioned into 2^{n-k} (equal-sized) disjoint k -cubes. Suppose that S_k is the set of the disjoint k -cubes, denoted by $C_1^k, C_2^k, \dots, C_{|S_k|}^k$, on that procedure *Paths0* was executed, where $2 \leq k \leq n$ and $|S_k|$ denotes the number of k -cubes in S_k . It should be noted that if d_i (or substitution for d_i) is not included in a cube for all $1 \leq i \leq m$, then *Paths0* will not be executed on the cube. Let $T(C_h^k)$ denote the time complexity of executing *Paths0* (excluding the recursive calls) on C_h^k , and $T(S_k) = T(C_1^k) + T(C_2^k) + \dots + T(C_{|S_k|}^k)$, where $1 \leq h \leq |S_k|$. Moreover, we assume that C_h^k contains m_h^k destination nodes for the input of *Paths0*. We have $m = m_1^k + m_2^k + \dots + m_{|S_k|}^k$ because $C_1^k, C_2^k, \dots, C_{|S_k|}^k$ are disjoint to each other. Since C_1^n is the original n -cube, we have $T(C_1^n) = O(m)$ as shown above. With the same arguments, we have $T(C_h^k) = O(m_h^k)$ for all $1 \leq h \leq |S_k|$, which implies $T(S_k) = T(C_1^k) + T(C_2^k) + \dots + T(C_{|S_k|}^k) = O(m_1^k) + O(m_2^k) + \dots + O(m_{|S_k|}^k) = O(m_1^k + m_2^k + \dots + m_{|S_k|}^k) = O(m)$. As a result, the time complexity of *Paths0*(Φ, m, n, D, I) is $T(S_n) + T(S_{n-1}) + \dots + T(S_2) = (n-1) \times O(m) = O(mn)$, which is optimal in the sense that any procedure must take at least $O(mn)$ time in the worst case to construct m disjoint paths in an n -cube. It should be noted that *Paths0*(Φ, m, n, D, I) takes optimal $O(mn)$ space to store $DL(d_i)$ and P_i (consisting of at most n dimension numbers) for all $1 \leq i \leq m$, and so does *Paths1*(Φ, m, n, D, I).

In [25], a one-to-one mapping $\Omega: \{d_1, d_2, \dots, d_m\} \rightarrow \{1, 2, \dots, n\}$ was referred to as a *partial routing function*, where $m \leq n$. Obviously, we can obtain a routing function Φ by defining $I = \{\Omega(d_1), \Omega(d_2), \dots, \Omega(d_m)\}$ and $\Phi(d_i) = \Omega(d_i)$ for all $1 \leq i \leq m$, and we have $d_{i, \Phi(d_i)} = d_{i, \Omega(d_i)} = 1$ if $d_{i, \Omega(d_i)} = 1$. By Lemma 1 and the above results, we have the following theorem.

Theorem 1. Given non-origin nodes $d_1, d_2, \dots, d_m$ in an n -dimensional hypercube and a partial routing function Ω such that $d_{i, \Omega(d_i)} = 1$ for all $1 \leq i \leq m$, then

m node-disjoint shortest paths from origin to $d_1, d_2, \dots, d_m$, respectively, can be constructed in optimal $O(mn)$ time, where $m \leq n$.

It should be noted that Theorem 1 can be applied to arbitrary single source node and multiple destination nodes because hypercubes are node symmetric. Obviously, Theorem 1 provides a sufficient condition for the existence of m disjoint shortest paths, i.e., if there exists a partial routing function Ω such that $d_{i, \Omega(d_i)} = 1$ for all $1 \leq i \leq m$, then there exist m disjoint shortest paths from s to $d_1, d_2, \dots, d_m$, respectively. In fact, the necessary condition is identical to the sufficient condition, as explained below. Suppose that there exist m disjoint shortest paths $Q_1, Q_2, \dots, Q_m$ from s to $d_1, d_2, \dots, d_m$, respectively. Define $\Omega(d_i) = k$ if e_k is the immediate successor of s in Q_i , where $1 \leq i \leq m$ and $1 \leq k \leq n$. We have $\Omega(d_1), \Omega(d_2), \dots, \Omega(d_m)$ distinct to each other because $e_{\Omega(d_1)}, e_{\Omega(d_2)}, \dots, e_{\Omega(d_m)}$ are mutually distinct. Since Q_i is shortest, we have $d_{i,k} = 1$, which implies $d_{i, \Omega(d_i)} = d_{i,k} = 1$. Hence, we have a partial routing function Ω such that $d_{i, \Omega(d_i)} = 1$ for all $1 \leq i \leq m$. For arbitrary $d_1, d_2, \dots, d_m$, the existence of a partial routing function Ω such that $d_{i, \Omega(d_i)} = 1$ for all $1 \leq i \leq m$ can be verified in $O(mn^{1.5})$ time, as explained below.

Define $\Sigma_\Omega = \sum_{i=1}^m d_{i, \Omega(d_i)}$. A partial routing function

Ω is said to be *maximal* if $\Sigma_\Omega \geq \Sigma_{\Omega'}$ for any $\Omega': \{d_1, d_2, \dots, d_m\} \rightarrow \{1, 2, \dots, n\}$. An intuitive meaning of a maximal Ω is that a Ω having $d_{i, \Omega(d_i)} = 1$ as many as possible is desired. Obviously, if Ω is maximal with $\Sigma_\Omega < m$, then there does not exist a partial routing function Ω' such that $d_{i, \Omega'(d_i)} = 1$ for all $1 \leq i \leq m$ because otherwise $\Sigma_{\Omega'} = m > \Sigma_\Omega$, which contradicts that Ω is maximal. On the other hand (Ω is maximal with $\Sigma_\Omega = m$), we have a partial routing function Ω such that $d_{i, \Omega(d_i)} = 1$ for all $1 \leq i \leq m$. In the following, we show that a maximal Ω can be obtained in $O(mn^{1.5})$ time by solving a corresponding maximum bipartite matching problem.

Suppose $G = (U, V, E)$ is a bipartite graph, where U and V are two partite sets of vertices and E is the set of edges. A subset of E forms a *matching* in G if no two of them share a common vertex. A matching in G is *maximum* if the number of its edges is maximum. A maximum matching in G can be found in $O(|E| \sqrt{|U \cup V|})$ time by Hopcroft and Karp (see [12]). Let $U = \{d_1, d_2, \dots, d_m\}$, $V = \{1, 2, \dots, n\}$, and $E = \{(d_i, k) \mid d_{i,k} = 1, 1 \leq i \leq m, \text{ and } 1 \leq k \leq n\}$. Suppose that a maximum matching M in G contains c edges, where $0 \leq c \leq m$. We first define $\Omega: \{d_1, d_2, \dots, d_m\} \rightarrow$

$\{1, 2, \dots, n\}$ as follows: $\Omega(d_i)=k$ if $(d_i, k) \in M$, and $\Omega(d_i)=0$ otherwise, where $1 \leq i \leq m$ and $1 \leq k \leq n$. Without loss of generality, we assume that $\Omega(d_i)=k_i$ for all $1 \leq i \leq c$ and $\Omega(d_j)=0$ for all $c+1 \leq j \leq m$.

Then, for all $c+1 \leq j \leq m$, define $\Omega(d_j)=k_j$ such that $k_{c+1}, k_{c+2}, \dots, k_m$ are distinct to each other and $k_j \in \{1, 2, \dots, n\} - \{k_1, k_2, \dots, k_c\}$. We have $\Sigma_{\Omega}=c$, as explained below. Suppose conversely that $\Sigma_{\Omega} \neq c$. By definition and assumption, we have $d_{i, \Omega(d_i)} =$

$d_{i, k_i} = 1$ for all $1 \leq i \leq c$. Since $c \neq \Sigma_{\Omega} = \sum_{i=1}^m d_{i, \Omega(d_i)} =$

$\sum_{i=1}^c d_{i, \Omega(d_i)} + \sum_{i=c+1}^m d_{i, \Omega(d_i)} = c + \sum_{i=c+1}^m d_{i, \Omega(d_i)}$, there

exists a $c+1 \leq w \leq m$ such that $d_{w, \Omega(d_w)} = 1$, which implies $(d_w, \Omega(d_w)) = (d_w, k_w) \in E$. Since $d_w \notin \{d_1, d_2, \dots, d_c\}$ and $k_w \notin \{k_1, k_2, \dots, k_c\}$, $M' = M \cup \{(d_w, k_w)\}$ is a matching with $c+1$ edges, which contradicts that M is maximum. We also claim that $\Sigma_{\Omega}=c$ is maximized. If Σ_{Ω} is not maximized, then there exist a one-to-one mapping $\Omega' : \{d_1, d_2, \dots, d_m\} \rightarrow \{1, 2, \dots, n\}$ such that $\Sigma_{\Omega'} = c' > c$. Define a matching M' as $\{(d_i, \Omega'(d_i)) \mid d_{i, \Omega'(d_i)} = 1 \text{ and } 1 \leq i \leq m\}$. It follows that M' contains c' edges, which contradicts to that M is maximum.

In summary, a maximal partial routing function $\Omega : \{d_1, d_2, \dots, d_m\} \rightarrow \{1, 2, \dots, n\}$ can be obtained in $O(mn \sqrt{m+n}) = O(mn^{1.5})$ time by finding a maximum matching in a bipartite graph $G=(U, V, E)$, where $U=\{d_1, d_2, \dots, d_m\}$, $V=\{1, 2, \dots, n\}$, and $E=\{(d_i, k) \mid d_{i, k}=1, 1 \leq i \leq m, \text{ and } 1 \leq k \leq n\}$.

3 Experiment results

In order to figure out the probability that there exist disjoint shortest paths from s to $d_1, d_2, \dots, d_m$ in an n -cube, an experiment was conducted by the aids of running computer programs, where $m \leq n$ and s was assumed to be the origin, i.e. 0^n . In this section, we outline the process of the experiment and describe its results.

In order to make the experiment not only representative but accomplished within a reasonable short time, it was carried out in the situation that all of the source node s and destination nodes $d_1, d_2, \dots, d_n$ are mutually distinct for $4 \leq n \leq 7$. In the experiment, $C(2^n-1, n)$ different combinations of $d_1, d_2, \dots, d_n$ were first enumerated in a recursive way by executing a computer program. For the interested reader, please visit URL "http://owm.nkmu.edu.tw/public/optimal.c" for the computer program, which was coded in C language and can be compiled by any ANSI C/C++ compiler. Then, for each combination, a maximal partial routing function $\Omega : \{d_1, d_2, \dots, d_n\} \rightarrow \{1, 2, \dots, n\}$

was computed. If $\Sigma_{\Omega} = n$, then we have $d_{i, \Omega(d_i)} = 1$ for all $1 \leq i \leq n$, which implies that n disjoint shortest paths from s to $d_1, d_2, \dots, d_n$, respectively, can be constructed by Theorem 1. On the other hand ($\Sigma_{\Omega} < n$), there does not exist such disjoint shortest paths.

Refer to Table 1, the numbers of combinations for the existence of disjoint shortest paths (from s to $d_1, d_2, \dots, d_n$) are shown for $n=4, 5, 6$, and 7 . According to the experiment results, the probability of the existence of such disjoint shortest paths is greater than 87%, 89%, 91%, and 94% for $n=4, 5, 6$, and 7 , respectively. It follows that Theorem 1 is not only theoretically interesting but also useful in practical applications. Another interesting phenomenon is that the probability becomes higher as n gets larger, which means that Theorem 1 is more applicable and hence $Paths1(\Phi, m, n, D, I)$ is more useful when n is large.

n	Total (A)	Existence of disjoint shortest paths (B)	Probability (B/A)
4	1,365	1,201	87.98%
5	169,911	151,286	89.03%
6	67,945,521	62,453,670	91.91%
7	89,356,415,775	84,707,326,890	94.79%

Table 1. The summary of experiment results

4 Concluding remarks

In this paper, we focus on a special problem of constructing m disjoint shortest paths from one source node to other m (not necessarily distinct) destination nodes in an n -cube, provided that there exist such disjoint shortest paths, where $m \leq n$. By the aids of routing functions, an optimal construction procedure $Paths1(\Phi, m, n, D, I)$ was proposed for constructing m disjoint shortest paths from s to $d_1, d_2, \dots, d_m$, respectively, in an n -cube, provided the existence of such disjoint shortest paths which can be verified in $O(mn^{1.5})$ time. In the situation that $s, d_1, d_2, \dots, d_m$ are mutually distinct, experiment results show that the probability of the existence of such disjoint shortest paths is greater than 87%, 89%, 91%, and 94% for $m=n=4, 5, 6$, and 7 , respectively. It follows that $Paths1(\Phi, m, n, D, I)$ is not only theoretically interesting but also useful in practical applications.

In addition, there are other applications of $Paths1(\Phi, m, n, D, I)$. The first application is that by

taking advantages of the optimal efficiency of $Paths1(\Phi, m, n, D, I)$, m disjoint paths from s to $d_1, d_2, \dots, d_m$, respectively, with their total length minimized in an n -cube can be constructed in $O(mn^{1.5} + m^3n)$ time [22], which is more efficient than $O(m^2n^{2.5} + mn^3)$ time of [13]. Besides, their maximal length is also minimized in the worst case. The second application of $Paths1(\Phi, m, n, D, I)$ is that the maximal length of disjoint paths (from s to $d_1, d_2, \dots, d_m$) obtained in [13] can be further reduced by at least one if one of two conditions hold [23]. It was shown in [23] that the probability that one of the two conditions holds is greater than 85% for $m=n=7$ and mutually distinct $s, d_1, d_2, \dots, d_n$. Other applications of $Paths1(\Phi, m, n, D, I)$, such as improving the efficiency of constructing disjoint paths in folded hypercube networks, are also under study.

References

- [1] B. Bollobás, *Extremal graph theory*, Academic Press, New York, 1978.
- [2] J. A. Bondy and U. S. R. Murty, *Graph theory with applications*, North-Holland, New York, 1976.
- [3] C. C. Chen and J. Chen, "Nearly optimal one-to-many parallel routing in star networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 8, no. 12, pp. 1196-1202, 1997.
- [4] X. B. Chen, "Many-to-many disjoint paths in faulty hypercubes," *Information Sciences*, vol. 179, no. 18, pp. 3110-3115, 2009.
- [5] E. Cheng, S. Gao, K. Qiu, and Z. Shen, "On disjoint shortest paths routing on the hypercube," *Proceedings of the third Annual International Conference on Combinatorial Optimization and Applications (COCO'A'09)*, LNCS 5573, 2009, pp. 375-383.
- [6] K. Day and A. E. Al-Ayyoub, "Fault diameter of k -ary n -cube networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 8, no. 9, pp. 903-907, 1997.
- [7] M. Dietzfelbinger, S. Madhavapeddy, and I. H. Sudborough, "Three disjoint path paradigms in star networks," *Proceedings of the third IEEE Symposium on Parallel and Distributed Processing*, 1991, pp. 400-406.
- [8] D. R. Duh and G. H. Chen, "On the Rabin number problem," *Networks*, vol. 30, no. 3, pp. 219-230, 1997.
- [9] D. R. Duh, G. H. Chen, and D. F. Hsu, "Combinatorial properties of generalized hypercube graphs," *Information Processing Letters*, vol. 57, no. 1, pp. 41-45, 1996.
- [10] A. El-Amawy and S. Latifi, "Properties and performance of folded hypercubes," *IEEE Transactions on Parallel and Distributed Systems*, vol. 2, no. 1, pp. 31-42, 1991.
- [11] J. S. Fu, "Longest fault-free paths in hypercubes with vertex faults," *Information Science*, vol. 176, pp. 759-771, 2006.
- [12] Z. Galil, "Efficient algorithms for finding maximum matching in graphs," *ACM Computing Surveys*, vol. 18, no. 1, pp. 23-38, 1986.
- [13] S. Gao, B. Novick, and K. Qiu, "From Hall's matching theorem to optimal routing on hypercubes," *Journal of Combinatorial Theory, Series B*, vol. 74, pp. 291-301, 1998.
- [14] T. F. Gonzalez and D. Serena, " n -Cube network: node disjoint shortest paths for maximal distance pairs of vertices," *Parallel Computing*, vol. 30, no. 8, pp. 973-998, 2004.
- [15] Q. P. Gu and S. Peng, "An efficient algorithm for the k -pairwise disjoint paths problem in hypercubes," *Journal of Parallel and Distributed Computing*, vol. 60, pp. 764-774, 2000.
- [16] Q. P. Gu and S. Peng, " k -Pairwise cluster fault tolerant routing in hypercubes," *IEEE Transactions on Computers*, vol. 46, no. 9, pp. 1042-1049, 1997.
- [17] Q. P. Gu and S. Peng, "Node-to-set and set-to-set cluster fault tolerant routing in hypercubes," *Parallel Computing*, vol. 24, pp. 1245-1261, 1998.
- [18] S. Y. Hsieh and N. W. Chang, "Extended fault-tolerant cycle embedding in faulty hypercubes," *IEEE Transactions on Reliability*, vol. 58, no. 4, pp. 702-710, 2009.
- [19] S. Y. Hsieh and Y. F. Weng, "Fault-tolerant embedding of pairwise independent Hamiltonian paths on a faulty hypercube with edge faults," *Theory of Computing Systems*, vol. 45, no. 2, pp. 407-425, 2009.
- [20] D. F. Hsu, "On container width and length in graphs, groups, and networks," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Science*, vol. E77-A, no. 4, pp. 668-680, 1994.
- [21] C. N. Lai, "One-to-Many Disjoint Paths in the Hypercube and Folded Hypercube," Ph.D. thesis, Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan, 2001.
- [22] C. N. Lai, "Optimal construction of all shortest node-disjoint paths in hypercubes with applications," submitted to *IEEE Transactions on Parallel and Distributed Systems*, 2011.
- [23] C. N. Lai, "Two conditions for reducing the maximal length of node-disjoint paths in hypercubes," submitted to *Theoretical Computer Science*, 2011.
- [24] C. N. Lai, G. H. Chen, and D. R. Duh, "Constructing one-to-many disjoint paths in folded hypercubes," *IEEE Transactions on*

- Computers*, vol. 51, no. 1, pp. 33-45, 2002.
- [25] C. N. Lai and G. H. Chen, " w -Rabin numbers and strong w -Rabin numbers of folded hypercubes," *Networks*, vol. 51, no. 3, pp. 171-177, 2008.
- [26] S. Latifi, H. Ko, and P. K. Srimani, "Node-to-set vertex disjoint paths in hypercube networks," Technical Report CS-98-107, Colorado State University, 1998.
- [27] S. C. Liaw and G. J. Chang, "Generalized diameters and Rabin numbers of networks," *Journal of Combinatorial Optimization*, vol. 4, pp. 371-384, 1999.
- [28] K. Qiu, "An efficient disjoint shortest paths routing algorithm for the hypercube," *Proceedings of the 14th IEEE International Conference on Parallel and Distributed Systems (ICPADS 2008)*, IEEE Computer Society Press, Melbourne, 2008, pp. 43-47.
- [29] M. O. Rabin, "Efficient dispersal of information for security, load balancing, and fault tolerance," *Journal of the ACM*, vol. 36, no. 2, pp. 335-348, 1989.
- [30] Y. Saad and M. H. Schultz, "Topological properties of hypercubes," *IEEE Transactions on Computers*, vol. 37, no. 7, pp. 867-872, 1988.
- [31] D. Wang, "A low-cost fault-tolerant structure for the hypercube," *The Journal of Supercomputing*, Vol. 20, no. 3, pp. 203-216, 2001.
- [32] R. Y. Wu, G. H. Chen, Y. L. Kuo, and G. J. Chang, "Node-disjoint paths in hierarchical hypercube networks," *Information Sciences*, vol. 177, no. 19, pp. 4200-4207, 2007.
- [33] J. S. Yang, S. M. Tang, J. M. Chang, and Y. L. Wang, "Parallel construction of optimal independent spanning trees on hypercubes," *Parallel Computing*, vol. 33, pp. 73-79, 2007.